

CPS 698 Capstone Project Proposal

TEAM — 1

Trinadh Musunuri - musun1t

Himasri Valli Tatineni - tatin1h

Yuyan Wang - wang39y

DataCrew: A Multi-Agent AI System for Safe, Conversational Database Analysis

Introduction

Natural-language-to-SQL systems are becoming a common way for non-technical users to interact with databases, but most existing implementations use a single large language model (LLM) to generate and execute a query in one uninterrupted pass. This design has no independent verification step, meaning ambiguous or destructive requests (e.g., "clean up old records") can silently translate into harmful operations, and users are given answers with no visibility into how those answers were produced. This project, DataCrew, proposes a multi-agent AI system in which a coordinated team of specialized, containerized agents — connection, query generation, safety verification, statistical analysis, critique, and reporting — work together to safely query a live database, analyze the results, and explain their reasoning to the user in real time. Rather than trusting a single model end to end, DataCrew applies a generate-then-verify pattern, where one agent's output is independently checked by another before it reaches the user or the database.

The primary computer science areas of focus for this project include Multi-Agent Systems and AI Orchestration, Natural Language Processing, Databases and Data Management, and Software Security.

- **Multi-Agent Systems and AI Orchestration:** A coordinator manages a pipeline of specialized LLM agents, each with a distinct role and scope, communicating through a defined message/state protocol rather than a single monolithic prompt.
- **Natural Language Processing:** LLM agents translate user questions in plain English into structured database queries (SQL/NoSQL) and translate structured results back into plain-English explanations.
- **Databases and Data Management:** The system connects to and queries live relational and non-relational databases (PostgreSQL, MySQL, MongoDB), handling schema introspection, query execution, and result aggregation at scale.
- **Software Security:** A dedicated Safety Agent verifies proposed database operations before they run, backed by a read-only credential and query-timeout enforced at the MCP database connector — MCP tool calls are structured and predefined, so there is no arbitrary code to sandbox at that layer. A separate, network-isolated sandbox container is used specifically to run the Analysis Agent's generated Python analysis code, which is a distinct and genuine arbitrary-code-execution risk.

Tools/Applications/Systems

Several tools and technologies are critical for implementing DataCrew's multi-agent orchestration, database connectivity, and sandboxed analysis pipeline:

- LangGraph / CrewAI: Frameworks for coordinating multiple specialized LLM agents, managing state, and handling agent-to-agent handoffs within the pipeline.
- Claude API (Anthropic): LLM backend for all agents, used with function/tool calling for structured query generation and safety/critique reasoning.
- Model Context Protocol (MCP) Database Servers: Standardized, containerized connectors for PostgreSQL, MySQL, and MongoDB that expose schema introspection and query execution as agent-callable tools.
- Docker / Docker Compose: Containerizes each agent and each database engine's MCP server. A separate, network-isolated sandbox container runs only the Analysis Agent's generated Python analysis code (pandas, NumPy, statsmodels) — database queries themselves execute directly through the scoped, read-only MCP connector, not inside this sandbox.
- FastAPI: Backend gateway coordinating agent calls and streaming live agent-trace updates to the frontend via WebSocket.
- React / Next.js: Frontend chat interface, live agent-trace panel, and interactive chart/report display.
- Plotly / Matplotlib: Chart generation for the Analysis and Reporting agents.
- Pandas, NumPy, SciPy, statsmodels: Statistical computation libraries used by the Analysis Agent inside the sandboxed container.

Data Sources/References

The project will use publicly available sample databases to seed the test environment, along with a benchmark of adversarial prompts and statistical test cases constructed by the team:

- Sample relational databases (e.g., the PostgreSQL "pagila" DVD-rental sample database, the MySQL "Sakila" sample database) used to seed containerized test instances with realistic schemas and data.
- Publicly available tabular datasets (e.g., Kaggle datasets) loaded into MongoDB/Postgres containers to test cross-schema and semi-structured querying.
- A team-constructed benchmark of 50–100 adversarial or ambiguous natural-language prompts (e.g., "clean up old records," "remove inactive users") used to evaluate the Safety Agent's catch rate and false-positive rate.
- A team-constructed set of 20–30 "statistical trap" questions/datasets (e.g., Simpson's paradox scenarios, non-normal distributions, small-sample spurious correlations) used to evaluate the Critic Agent.

References

Suresh Babu, R., & Agrawal, A. (2026). Self-Healing Agentic Orchestrators for Reliable Tool-Augmented Large Language Model Systems. arXiv:2606.01416.

Tan, J., Lin, H., Guo, S., Fang, Y., Luo, X., Shen, T., & Qiao, Z. (2026). Safe and Adaptive Cloud Healing: Verifying LLM-Generated Recovery Plans with a Neural-Symbolic World Model. arXiv:2607.01595.

Pan, R., Zhang, H., & Liu, C. (2025). CodeCoR: An LLM-Based Self-Reflective Multi-Agent Framework for Code Generation. arXiv:2501.07811.

Pappu, A., El, B., Cao, H., di Nolfo, C., Sun, Y., Cao, M., & Zou, J. (2026). Multi-Agent Teams Hold Experts Back. arXiv:2602.01011.

Dong, L., Lu, Q., & Zhu, L. (2024). AgentOps: Enabling Observability of LLM Agents. arXiv:2411.05285.

Summary

This project addresses the growing risk of unverified, single-agent natural-language database access, where an LLM generates and immediately executes queries with no independent check on correctness or safety. Existing research on self-healing and recovery-plan verification in agentic systems has largely targeted DevOps automation and general tool use, not conversational database analysis specifically — leaving an open gap in how AI-generated data operations can be safely trusted. DataCrew addresses this gap by decomposing the query-to-answer pipeline across specialized agents, most notably a Safety Agent that verifies proposed database operations before execution and a Critic Agent that reviews statistical analyses for common errors before they reach the user.

The core research question is whether this explicit verification pipeline meaningfully improves reliability measured as the catch rate of unsafe or destructive queries and statistically invalid conclusions compared to a single-pass agent baseline, and at what cost in added latency. This is evaluated empirically against a purpose-built benchmark of adversarial prompts and statistical "trap" datasets, directly engaging with an open and actively debated question in current multi-agent systems research: whether multi-agent decomposition provides a genuine reliability benefit over a single, well-prompted agent. The resulting system offers a real-world, demoable application safe, conversational access to live databases for non-technical users while the benchmark and evaluation results constitute a reusable research contribution beyond the software itself.